Lab Work 7: Setting up a Basic ASP.NET Core MVC Application

Objective:
Learn how to set up an ASP.NET Core MVC application and understand the project structure.

Lab Instructions:

- **Create a new ASP.NET Core MVC & ASP .NET API project** using both Visual Studio or .NET CLI:
- Write different commands for creating projects

Lab Work 8: Understanding Controllers and ActionResult in ASP.NET Core

The objective of this lab is to understand how to create controllers in ASP.NET Core MVC and work with different types of ActionResult return types to handle different HTTP responses such as views, redirects, JSON data, and file downloads.

Create a new controller named ProductController.

1. In the Index action of the ProductController, return a View that displays a list of products.
2. Add a new action GetProductInfo in ProductController that returns product information in JSON format.
3. Add a new action RedirectToHome in ProductController that returns a RedirectResult to the home page (i.e., Home/Index).
4. Add a new action DownloadProductFile in ProductController that returns a file for download. You can use a simple text file or image as the content.[File]
5. Add a new action DisplayMessage in ProductController that returns a simple string message using ContentResult.
6. Add a new action ReturnStatusCode in ProductController that returns a custom HTTP status code (e.g., 204 No Content).[StatusCodeResult]

Lab Work 9: Razor Syntax, Tag Helpers, and Custom Tag Helpers in ASP.NET Core MVC

Objective:
The objective of this lab is to understand how to use **Razor syntax**, apply **Tag Helpers** in ASP.NET Core MVC, and create **Custom Tag Helpers** to enhance the functionality and maintainability of views.

Task 1: Razor Syntax, ViewBag, and ViewData

Controller:

- Create RazorDemoController with an Index action.
- In the action, assign:
    - A string message to ViewBag.Message.
    - A list of integers to ViewData["Numbers"].

View (Index.cshtml):

- Display ViewBag.Message.
- Use @foreach to loop through ViewData["Numbers"] and display each number.
- Add a conditional statement:
    - If the number is greater than 10, display "Number is large".
    - Otherwise, display "Number is small".

Task 2: Tag Helper and Custom Tag Helper

1. **Built-in Tag Helpers:**
    - Create a simple form in Index.cshtml that uses asp-for tag helpers for input fields (Name, Email).
    - Use the asp-validation-for tag helper to display validation errors for these fields.
    - Add a submit button using asp-button tag helper.
2. **Custom Tag Helper:**
    - Create a custom Tag Helper DisplayDateTagHelper that takes a date parameter and formats it as "MM/dd/yyyy".
    - In the view, use the custom tag helper to display the current date using <display-date date="DateTime.Now"></display-date>.

Lab Work 10: Model Binding & Validations in ASP.NET Core MVC

Objective:
To understand and implement **Model Binding** and **Data Validation** in ASP.NET Core MVC, including how to bind form data to models and apply validation using data annotations.

Task 1: Model Binding

1. **Create a Model:**
    - Create a Person model with the following properties:
        - Name (string)
        - Email (string)
        - Age (int)
2. **Create a Controller (ModelBindingController):**
    - Add an Index action to render a form for creating a Person object.
    - Add a Create action to handle the form submission and bind the form data to a Person model using **Model Binding**.
3. **Create a View (Index.cshtml):**
    - Create a form that uses asp-for tag helpers to bind input fields for Name, Email, and Age to the Person model.
    - Use the asp-action attribute to submit the form to the Create action.

Task 2: Data Validation with Annotations

1. **Add Data Annotations to the Person Model:**
    - Add validation annotations to the properties of the Person model:
        - Name: Required ([Required]), MaxLength ([MaxLength(50)]).
        - Email: Required ([Required]), Email address format ([EmailAddress]).
        - Age: Required ([Required]), Range between 18 and 100 ([Range(18, 100)]).
2. **Update the Controller (Create action):**
    - In the Create action, check if the model is valid using ModelState.IsValid. If it is not valid, return to the form with validation errors.
3. **Display Validation Errors in the View:**
    - Use asp-validation-for tag helpers in the view to display validation messages for Name, Email, and Age.
4. **Submit the Form:**
    - Test the form by submitting invalid data (e.g., empty fields, invalid email, or age out of range).
    - Ensure the validation messages are displayed when invalid data is submitted.